

Week 6: Interview Prep Pack

CI/CD (GitHub Actions / GitLab CI): Core Concepts

Q1. Explain what a CI pipeline does.

A CI pipeline automatically builds, tests, and validates code changes every time code is pushed.

Ensures:

- code compiles
- tests pass
- Docker/Terraform changes are correct

Q2. What are the common stages in a CI pipeline?

- **Build:** compile code, build Docker image
- **Test:** unit tests, functional tests
- **Security:** scanning, linting
- **Validate:** Terraform plan, schema checks
- **Package:** push artifact

Q3. How do you store secrets in GitHub Actions/GitLab CI?

Using:

- GitHub Secrets
- GitLab Variables (masked, protected)

Never commit secrets.

Q4. What is the purpose of caching in CI pipelines?

- Speeds up builds
- Reuses Terraform modules
- Reuses Docker layers

Q5. How do you trigger pipelines?

- push
- pull_request
- schedule
- manual workflow dispatch

Q6. Your CI pipeline fails with “terraform init backend error.” What do you check?

- Missing backend config
- Missing cloud credentials
- Wrong bucket name
- IAM permissions

Q7. Your Docker builds take too long. How do you optimise?

- Use smaller base images
- Use multi-stage builds
- Reorder layers for caching
- Use registry caching

Q8. CI pipeline passes locally but fails in GitHub Actions. Why?

- Missing dependencies
- Different OS/architecture
- Environment-specific variables
- File path differences (case-sensitive)

DevSecOps (SBOMs, Vulnerability Scanning, Image Signing)

Q1. What is an SBOM and why is it important?

SBOM = Software Bill of Materials

Lists all packages & dependencies inside an artifact.

Used for:

- compliance
- vulnerability lookup
- supply chain transparency

Q2. What tools generate SBOMs?

- Syft
- Docker Scout
- Trivy

Q3. What is container vulnerability scanning?

Scanning Docker images for CVEs (Common Vulnerabilities & Exposures).

Tools:

- Trivy
- Gype

Q4. What is image signing? Why do we sign images?

Image signing ensures:

- authenticity
- integrity
- provenance

Cosign is the modern tool.

Q5. Your pipeline detects CRITICAL vulnerabilities. What do you do?

Depends on severity:

- Patch base image
- Upgrade package
- Add temporary ignore (if risk accepted)
- Create Jira ticket (if not blocking)

Q6. Your signed image fails verification in cluster. Why?

- Wrong public key
- Key not trusted by admission controller
- Signature stored in wrong registry location
- Tag mismatch

Q7. Your SBOM shows outdated libraries. Who is responsible?

DevSecOps ≠ blocking everything

Developers + platform + security collaborate.



GitOps & ArgoCD: Core Interview Concepts

Q1. What is GitOps?

GitOps = Git as the single source of truth for deployments.
Cluster state is automatically reconciled with Git.

Q2. Pull-based vs Push-based deployment?

- **Push-based:** CI pushes manifests to cluster
- **Pull-based (GitOps):** ArgoCD pulls from Git

GitOps is safer and auditable.

Q3. What is drift in GitOps?

When Kubernetes resources deviate from Git definitions.
Example: manual kubectl edits.

Q4. How does ArgoCD detect drift?

- Compares live cluster state with Git commit
- Flags OutOfSync
- Shows diff in UI

Q5. What happens when auto-sync is enabled?

ArgoCD immediately attempts to restore resources to match Git.

Q6. The developer edited a Deployment manually. ArgoCD didn't fix it. Why?

Possible reasons:

- Auto-sync disabled
- Annotation added to ignore changes
- ArgoCD service account lacks RBAC
- Manifest missing in Git

Q7. ArgoCD shows SyncError. What do you check?

- Wrong repo path
- YAML syntax errors
- Missing CRDs
- Access to private repo
- Cluster RBAC

Q8. Deployment in Git shows 5 replicas, but cluster shows 2. Why?

- Manual override
- ArgoCD not synced
- Auto-sync paused due to sync failures
- Health check failed

Internal Developer Platforms (IDPs), Backstage & Golden Paths

Q1. What is an Internal Developer Platform (IDP)?

A productized platform that gives developers:

- reusable templates
- self-service tools
- golden paths
- consistent workflows

Q2. What is Backstage used for?

- Service catalog
- Documentation hub
- Templates (Scaffolder)
- Developer workflows

Q3. What is a Golden Path?

A recommended, secure, standardized way for developers to ship applications.

Q4. What are scorecards?

Automated checks that validate:

- CI present
- SBOM present
- Security scans done
- Dockerfile best practices

Q5. Developers complain pipelines are inconsistent. How do you fix this using a platform?

- Provide golden path templates
- Provide example repo structure
- Add scaffolder templates
- Add scorecards

Q6. Developer onboarding takes too long. How do you solve using Backstage?

- Auto-generate new service repos
- Link documentation
- Provide sample CI/CD pipeline
- Provide secure-by-default templates

Q7. Platform adoption is low. What do you do?

- Interview developers
- Identify friction
- Improve UX
- Reduce mandatory steps
- Provide self-service and automation

Hands-On Debugging Questions

Q1. Terraform plan works locally but fails in CI. Why?

- Missing backend creds
- Remote backend buckets unreachable
- Missing Terraform plugins
- CI uses different version

Q2. Your Docker build fails in CI with “file not found.” Why?

- Wrong working directory
- Case-sensitive path in Linux
- Missing COPY path

Q3. ArgoCD sync is stuck. What do you check?

- Health checks
- Resource finalizers
- CRDs missing
- RBAC permission denied

Q4. Vulnerability scan is blocking pipeline. What do you do?

- Evaluate risk
- Upgrade dependencies
- Use base image with fewer CVEs
- Set threshold (block only critical)

Q5. Signed image not deployable. How to debug?

- Validate signature via cosign verify
- Admission controller logs
- Registry permissions
- Tag mismatch

Senior-Level Conceptual Questions

Q1. What is “shift left” in security?

THE DEVOPS WAR ROOM
Move security early into CI/CD pipeline → not after deployment.

Q2. How do you design a secure CI pipeline?

- Least privilege
- Secrets stored securely
- Infrastructure validation
- Image scanning
- Signing
- Policy checks (OPA/Gatekeeper)

Q3. What is the main benefit of GitOps compared to traditional CI/CD?

- Deterministic deployments
- Auto-healing
- Auditable changes
- Strong separation of duties

Q4. What's the difference between platform engineering and DevOps?

DevOps = philosophy

Platform engineering = a team that builds tools enabling DevOps

